

Name: \_\_\_\_\_





# BOOTSTRAP

Equity • Scale • Rigor

Brought to you by the Bootstrap team:

- Emmanuel Schanzer
- Kathi Fiser
- Shriram Krishnamurthi
- Dorai Sitaram
- Joe Politz
- Ben Lerner
- Nancy Pfenning
- Flannery Denny
- Rachel Tabak

# Introduction to Programming in a Nutshell

The **Editor** is a software program we use to write Code. Our Editor allows us to experiment with Code on the right-hand side, in the **Interactions Area**. For Code that we want to *keep*, we can put it on the left-hand side in the **Definitions Area**. Clicking the "Run" button causes the computer to re-read everything in the Definitions Area and erase anything that was typed into the Interactions Area.

## Data Types

Programming languages involve different **data types**, such as Numbers, Strings, Booleans, and even Images.

- Numbers are values like `1`, `7/8`, and `-8261.003`.
  - Numbers are *usually* used for quantitative data and other values are *usually* used as categorical data.
  - In Pyret, decimals *must* start with a zero. For example, `0.22` is valid, but `.22` is not.
- Strings are values like `"Emma"`, `"Rosanna"`, `"Jen and Ed"`, or even `"08/28/1980"`.
  - All strings *must* be surrounded by quotation marks.
- Booleans are either `true` or `false`.

All values evaluate to themselves. The program `42` will evaluate to `42`, the String `"Hello"` will evaluate to `"Hello"`, and the Boolean `false` will evaluate to `false`.

## Operators

Operators (like `+`, `-`, `*`, `<`, etc.) work the same way in Pyret that they do in math.

- Operators are written between values, for example: `4 + 2`.
- In Pyret, operators must always have spaces around them. `4 + 2` is valid, but `4+2` is not.
- If an expression has different operators, parentheses must be used to show Order of Operations. `4 + 2 + 6` and `4 + (2 * 6)` are valid, but `4 + 2 * 6` is not.

## Applying Functions

Functions work much the way they do in math. Every function has a name, takes some inputs, and produces some output. The function name is written first, followed by a list of **arguments** in parentheses.

- In math this could look like  $f(5)$  or  $g(10, 4)$ .
- In Pyret, these examples would be written as `f(5)` and `g(10, 4)`.
- Applying a function to make images would look like `star(50, "solid", "red")`.
- There are many other functions in Pyret, for example `sqr`, `sqrt`, `triangle`, `square`, `string-repeat`, etc.

Functions have **contracts**, which help explain how a function should be used. Every Contract has three parts:

- The *Name* of the function — literally, what it's called.
- The *Domain* of the function — what *type(s) of value(s)* the function consumes, and in what order.
- The *Range* of the function — what *type of value* the function produces.

# Strings and Numbers

Make sure you've loaded [\(pyret.BootstrapWorld.org\)](http://pyret.BootstrapWorld.org), clicked "Run", and are working in the **Interactions Area** on the right. Hit Enter/return to evaluate expressions you test out.

## Strings

String values are always in quotes.

- Try typing your name (in quotes!).
- Try typing a sentence like "I'm excited to learn to code!" (in quotes!).
- Try typing your name with the opening quote, but *without the closing quote*. Read the error message!
- Now try typing your name *without any quotes*. Read the error message!

1) Explain what you understand about how strings work in this programming language. \_\_\_\_\_

## Numbers

2) Try typing 42 into the Interactions Area and hitting "Enter". Is 42 the same as "42"? Why or why not?

\_\_\_\_\_

3) What is the largest number the editor can handle?

\_\_\_\_\_

4) Try typing .5. Then try typing 0.5. Then try clicking on the answer. Experiment with other decimals.

Explain what you understand about how decimals work in this programming language. \_\_\_\_\_

\_\_\_\_\_

5) What happens if you try a fraction like 1/3? \_\_\_\_\_

\_\_\_\_\_

6) Try writing **negative** integers, fractions and decimals. What do you learn? \_\_\_\_\_

\_\_\_\_\_

## Operators

7) Just like math, Pyret has **operators** like +, -, \*, and /.

Try typing in 4 + 2 and then 4+2 (without the spaces). What can you conclude from this?

\_\_\_\_\_

8) Type in the following expressions, **one at a time**: 4 + 2 \* 6    (4 + 2) \* 6    4 + (2 \* 6)    What do you notice?

\_\_\_\_\_

9) Try typing in 4 + "cat", and then "dog" + "cat". What can you conclude from this?

\_\_\_\_\_

\_\_\_\_\_

# Booleans

Boolean-producing expressions are yes-or-no questions, and will always evaluate to either **true** ("yes") or **false** ("no").

What will the expressions below evaluate to? Write down your prediction, then type the code into the Interactions Area to see what it returns.

| Prediction                    | Result | Prediction                        | Result |
|-------------------------------|--------|-----------------------------------|--------|
| 1) <code>3 &lt;= 4</code>     |        | 2) <code>"a" &gt; "b"</code>      |        |
| 3) <code>3 == 2</code>        |        | 4) <code>"a" &lt; "b"</code>      |        |
| 5) <code>2 &lt; 4</code>      |        | 6) <code>"a" == "b"</code>        |        |
| 7) <code>5 &gt;= 5</code>     |        | 8) <code>"a" &lt;&gt; "a"</code>  |        |
| 9) <code>4 &gt;= 6</code>     |        | 10) <code>"a" &gt;= "a"</code>    |        |
| 11) <code>3 &lt;&gt; 3</code> |        | 12) <code>"a" &lt;&gt; "b"</code> |        |
| 13) <code>4 &lt;&gt; 3</code> |        | 14) <code>"a" &gt;= "b"</code>    |        |

15) In your own words, describe what `<` does. \_\_\_\_\_

16) In your own words, describe what `>=` does. \_\_\_\_\_

17) In your own words, describe what `<>` does. \_\_\_\_\_

|                                                   | Prediction: | Result: |
|---------------------------------------------------|-------------|---------|
| 18) <code>string-contains("catnap", "cat")</code> |             |         |
| 19) <code>string-contains("cat", "catnap")</code> |             |         |

20) In your own words, describe what `string-contains` does. Can you generate another expression using `string-contains` that returns true?

★ There are infinite string values ("a", "aa", "aaa" ...) and infinite number values out there (...-2,-1,0,1,2...). But how many different *Boolean* values are there? \_\_\_\_\_

These materials were developed partly through support of the National Science Foundation (awards 1042210, 1535276, 1648684, and 1738598) and are licensed under a Creative Commons 4.0 Unported License. Based on a work at [www.BootstrapWorld.org](http://www.BootstrapWorld.org). Permissions beyond the scope of this license may be available by contacting [contact@BootstrapWorld.org](mailto:contact@BootstrapWorld.org).